

Jaguar Re-Identification

This project contains experiments performed while taking part in the Kaggle “[Jaguar Re-Identification Challenge](#)” where the task was to re-identify Jaguars from images. In practice, that means learning a representation that brings images of the same jaguar closer together and pushes images of different jaguars farther apart. We performed and documented several theme experiments that can be split into [Exploratory Data Analysis](#) (EDA) and [Leaderboard experiments](#). In our experiments, we start from a stable baseline, change one part of the pipeline that we want to study while keeping the rest fixed, and evaluate every step with identity-balanced mAP. For Leaderboard experiments with promising validation mAPs we generated a submission file which we uploaded to the Kaggle competition. The resulting public scores are documented as well.

Exploratory Data Analysis

One of the first things we noticed was a strong class imbalance: some jaguars appear in many images, while others have only a few samples. We tested weighted sampling to see whether this method can tackle this problem. The answer was only partly yes. Some rare identities benefited, but some even got worse, so the overall gain in identity-balanced mAP was very small.

After noticing varying results and training progress with similar parameters, we made sure to fix the random seed values in our experiments. This way we were able to reproduce our experiments reliably. We also spent time checking how stable our results really were by training the same pipeline on multiple seeds. The seed comparison experiment showed that performance varies noticeably across random initializations, which means that small improvements should be treated with caution unless they are repeatable. Another important finding came from the background analysis. The two Kaggle competitions provided two different datasets: one where background RGB values next to the jaguars were still present, and one where they were removed. To test the relevance of the background we trained two models with the same parameters on each of the data sources. The models performed similar when tested on test data of the same type (with background / without background). However, when cross validating the models we observed that the model trained on one version transferred poorly to the other. In other words, background handling is not just a small preprocessing detail in this project as it meaningfully changes the data distribution and needs to stay consistent.

Finally, we used an interpretability method (Integrated Gradients) to better understand what models are looking at. Notably we performed this analysis using a CNN model backbone (EfficientNet) while our other experiments are ViT based. With Integrated Gradients, plus a few sanity checks such as weight randomization and masking-based faithfulness tests, we could verify that this model was not just achieving good scores but was also relying on meaningful visual patterns in the jaguar images.

Leaderboard experiments

Our modeling strategy followed the same general idea: start simple, then improve the pipeline step by step. We first compared different backbones and found that large vision transformers, especially EVA-02 and DINOv3, worked much better than CNN backbones for this task. After that, we compared loss functions and saw that margin-based metric learning losses fit the retrieval objective better than standard cross-entropy. ArcFace turned out to be the most reliable choice and became the basis for all later experiments.

Once backbone and loss were fixed, we looked at the next big question: should we keep the backbone frozen, or fine-tune it end to end? This turned out to be one of the most effective measures. Full fine-tuning of the EVA-02 backbone clearly outperformed training only a small projection head on frozen embeddings. Based on that result, we moved on to a broad hyperparameter sweep covering learning rates, regularization, head dimensions, batch size, and augmentation, where we ultimately found a slightly better configuration.

We also tested a few refinements at inference time and in the architecture itself. K-reciprocal reranking was useful and could improve retrieval for suitable models, but the dedicated reranking analysis showed that the standard parameter setting was already difficult to beat. In contrast, deterministic test-time augmentation and using GeM pooling instead of the default mean pooling did not give robust gains in our setup. That is representative of how we worked throughout the project: we did not keep ideas just because they sounded plausible, but only if they showed a clear and repeatable benefit.